

Tarea 1 Bases de datos avanzadas

Andrés Jablonca

202173578-K

Tanto el dataset como las imágenes de este informe (entre otros archivos/resultados relevantes) se encuentran en el siguiente repositorio:

https://github.com/andres-jablonca/BDD_Avanzadas_2026-2/tree/main/t1

Preparación de los datos

Para el desarrollo de la tarea se generó un conjunto de datos ficticio correspondiente a actores y actrices de voz. La generación fue realizada mediante inteligencia artificial, utilizando como criterio principal obtener datos suficientemente variados para permitir la ejecución de todas las consultas requeridas posteriormente.

Se generaron **24 documentos**. Cada documento representa a un actor o actriz de voz e incluye información personal, antecedentes económicos, idiomas de doblaje y los personajes interpretados.

El conjunto de datos fue diseñado de manera intencional para incluir:

- actores y actrices mayores de 50 años;
- actores y actrices menores de 30 años;
- distintas nacionalidades;
- personas capaces de realizar doblaje en uno o más idiomas;
- valores variados de patrimonio;
- personajes principales, secundarios y extras;
- producciones profesionales e independientes;
- múltiples géneros, incluyendo acción, drama, fantasía, comedia, ciencia ficción, entre otros.

Esta diversidad permite que las consultas posteriores produzcan resultados significativos y que sea posible analizar adecuadamente el comportamiento de MongoDB.

Cada actor o actriz contiene los siguientes atributos principales:

Nombre

Nacionalidad

FechaNacimiento

Edad

Patrimonio

Idiomas

Personajes

En donde cada atributo está definido según lo especificado en las instrucciones.

Parte 1 - Consultas

1. *Proyección, filtrado y ordenamiento sobre campos simples. Realice una consulta que muestre únicamente actores o actrices de voz mayores de 50 años.*

The screenshot shows the MongoDB Compass interface. The query bar contains the query: `{edad:{$gt:50}}`. The Project field is set to `{_id:0,nombre:1,edad:1,nacionalidad:1,patrimonio:1}`. The Sort field is set to `{edad:-1}`. The Collation field is set to `{ locale: 'simple' }`. The Index Hint field is set to `{ field: -1 } or "indexName"`. The Max Time MS field is set to 60000. The Skip field is set to 0 and the Limit field is set to 10. The interface shows the results of the query in a table format. The table has 4 columns: nombre String, nacionalidad String, edad Int32, and patrimonio Int32. The results are as follows:

	nombre String	nacionalidad String	edad Int32	patrimonio Int32
1	"Michael Thompson"	"Estados Unidos"	66	8200000
2	"Roberto Mendoza"	"Mexico"	64	3500000
3	"Patricia Contreras"	"Chile"	60	2100000
4	"Ricardo Sepulveda"	"Chile"	58	1750000
5	"Diego Morales"	"Chile"	55	1200000
6	"Gabriela Torres"	"Argentina"	55	1450000
7	"Claire Dubois"	"Francia"	54	2850000
8	"Kenji Nakamura"	"Japon"	51	4600000

Figura 1: Captura que muestra la consulta 1 realizada en la interfaz gráfica junto con el resultado obtenido

2. *Búsqueda dentro de un arreglo simple. Seleccione una nacionalidad y un idioma existentes dentro de sus datos y realice una consulta que muestre actores o actrices de dicha nacionalidad que contengan ese idioma dentro del arreglo "Idiomas".*

The screenshot shows the MongoDB Compass interface. The query bar contains the query: `{nacionalidad:'Chile',idiomas:'Español'}`. The Project field is set to `{_id:0,nombre:1,nacionalidad:1,edad:1,idiomas:1}`. The Sort field is set to `{nombre:1}`. The Collation field is set to `{ locale: 'simple' }`. The Index Hint field is set to `{ field: -1 } or "indexName"`. The Max Time MS field is set to 60000. The Skip field is set to 0 and the Limit field is set to 10.

Figura 2: Captura que muestra la consulta 2 realizada en la interfaz gráfica.


```

{
  "nombre": "Aiko Tanaka",
  "nacionalidad": "Japon",
  "edad": 22,
  "personajes": [
    {
      "nombre": "Hikari",
      "produccion": "Blade Spirit",
      "tipoProduccion": "Profesional",
      "rol": "Principal",
      "apariciones": 49,
      "generos": [
        "Accion",
        "Anime",
        "Fantasia"
      ]
    }
  ]
}

{
  "nombre": "Luciana Fernandez",
  "nacionalidad": "Argentina",
  "edad": 23,
  "personajes": [ ]
}

{
  "nombre": "Emily Carter",
  "nacionalidad": "Estados Unidos",
  "edad": 24,
  "personajes": [ ]
}

{
  "nombre": "Valentina Rojas",
  "nacionalidad": "Chile",
  "edad": 25,
  "personajes": [ ]
}

{
  "nombre": "Daniela Castillo",
  "nacionalidad": "Mexico",
  "edad": 26,
  "personajes": [ ]
}

{
  "nombre": "Matias Fuentes",
  "nacionalidad": "Chile",
  "edad": 27,
  "personajes": [ ]
}

{
  "nombre": "Lucas Oliveira",
  "nacionalidad": "Brasil",
  "edad": 28,
  "personajes": [ ]
}

```

Figura 5: Captura que muestra el resultado de la consulta 3.

4. Pipeline de agregación. Utilizando “\$aggregate()”, muestre actores o actrices ordenados de acuerdo con su patrimonio. Deberá obtener 10 registros comenzando desde la quinta posición del resultado ordenado.

24 Documents in the collection

Preview of documents

Document 1	Document 2	Document 3	Document 4	Document 5
<pre> { "_id": ObjectId("6aa3864b805bfae6978233a5"), "nombre": "Valentina Rojas", "nacionalidad": "Chile", "fechaNacimiento": "2003-04-12", "edad": 25, "patrimonio": 180000, "idiomas": Array (2), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233a6"), "nombre": "Matias Fuentes", "nacionalidad": "Chile", "fechaNacimiento": "1999-08-20", "edad": 27, "patrimonio": 95000, "idiomas": Array (1), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233a7"), "nombre": "Daniela Soto", "nacionalidad": "Chile", "fechaNacimiento": "1989-02-14", "edad": 37, "patrimonio": 350000, "idiomas": Array (2), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233a8"), "nombre": "Diego Morales", "nacionalidad": "Chile", "fechaNacimiento": "1971-06-05", "edad": 55, "patrimonio": 1200000, "idiomas": Array (2), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233a9"), "nombre": "Patricia Contreras", "nacionalidad": "Chile", "fechaNacimiento": "1965-11-06", "edad": 60, "patrimonio": 2100000, "idiomas": Array (2), "personajes": Array (2) } </pre>

Stage 1: Sort

Output preview after Sort stage (Sample of 10 documents)

Document 1	Document 2	Document 3	Document 4
<pre> { "_id": ObjectId("6aa3864b805bfae6978233ab"), "nombre": "Luciana Fernandez", "nacionalidad": "Argentina", "fechaNacimiento": "2003-07-17", "edad": 23, "patrimonio": 75000, "idiomas": Array (2), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233ac"), "nombre": "Matias Fuentes", "nacionalidad": "Chile", "fechaNacimiento": "1999-08-20", "edad": 27, "patrimonio": 95000, "idiomas": Array (1), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233ad"), "nombre": "Daniela Castillo", "nacionalidad": "Mexico", "fechaNacimiento": "2000-03-28", "edad": 26, "patrimonio": 130000, "idiomas": Array (3), "personajes": Array (2) } </pre>	<pre> { "_id": ObjectId("6aa3864b805bfae6978233ae"), "nombre": "Aiko Tanaka", "nacionalidad": "Japon", "fechaNacimiento": "2004-01-22", "edad": 22, "patrimonio": 160000, "idiomas": Array (2), "personajes": Array (2) } </pre>

Figura 6: Captura que muestra la primera parte del pipeline de agregación realizado en la interfaz gráfica.

Stage 2

Skip

1

4

Output preview after **Skip** stage (Sample of 10 documents)

_id: ObjectId('6aa3864b805bfae6978233a5')

nombre: "Valentina Rojas"

nacionalidad: "Chile"

fechaNacimiento: "2001-04-12"

edad: 25

patrimonio: 180000

idiomas: Array (2)

personajes: Array (2)

_id: ObjectId('6aa3864b805bfae6978233b9')

nombre: "Lucas Oliveira"

nacionalidad: "Brasil"

fechaNacimiento: "1998-03-22"

edad: 28

patrimonio: 220000

idiomas: Array (2)

personajes: Array (1)

_id: ObjectId('6aa3864b805bfae6978233b2')

nombre: "Elena Martínez"

nacionalidad: "España"

fechaNacimiento: "1997-10-19"

edad: 28

patrimonio: 270000

idiomas: Array (3)

personajes: Array (2)

_id: ObjectId('6aa3864b805bfae6978233b6')

nombre: "Emily Carter"

nacionalidad: "Estados Ur"

fechaNacimiento: "2002-01-24"

edad: 24

patrimonio: 310000

idiomas: Array (2)

personajes: Array (2)

Stage 3

\$limit

1

10

Output preview after **\$limit** stage (Sample of 10 documents)

_id: ObjectId('6aa3864b805bfae6978233a5')

nombre: "Valentina Rojas"

nacionalidad: "Chile"

fechaNacimiento: "2001-04-12"

edad: 25

patrimonio: 180000

idiomas: Array (2)

personajes: Array (2)

_id: ObjectId('6aa3864b805bfae6978233b9')

nombre: "Lucas Oliveira"

nacionalidad: "Brasil"

fechaNacimiento: "1998-03-22"

edad: 28

patrimonio: 220000

idiomas: Array (2)

personajes: Array (1)

_id: ObjectId('6aa3864b805bfae6978233b2')

nombre: "Elena Martínez"

nacionalidad: "España"

fechaNacimiento: "1997-10-19"

edad: 28

patrimonio: 270000

idiomas: Array (3)

personajes: Array (2)

_id: ObjectId('6aa3864b805bfae6978233b6')

nombre: "Emily Carter"

nacionalidad: "Estados Ur"

fechaNacimiento: "2002-01-24"

edad: 24

patrimonio: 310000

idiomas: Array (2)

personajes: Array (2)

Stage 4

\$project

1

{

2

_id: 0,

3

nombre: 1,

4

edad: 1,

5

nacionalidad: 1,

6

patrimonio: 1

7

}

Output preview after **\$project** stage (Sample of 10 documents)

nombre: "Valentina Rojas"

edad: 25

patrimonio: 180000

nombre: "Lucas Oliveira"

edad: 28

patrimonio: 220000

nombre: "Elena Martínez"

edad: 28

patrimonio: 270000

nombre: "Emily Carter"

edad: 24

patrimonio: 310000

Figura 7: Captura que muestra la segunda parte del pipeline de agregación realizado en la interfaz gráfica.

```

< {
  nombre: 'Valentina Rojas',
  nacionalidad: 'Chile',
  edad: 25,
  patrimonio: 180000
}
{
  nombre: 'Lucas Oliveira',
  nacionalidad: 'Brasil',
  edad: 28,
  patrimonio: 220000
}
{
  nombre: 'Elena Martinez',
  nacionalidad: 'Espana',
  edad: 28,
  patrimonio: 270000
}
{
  nombre: 'Emily Carter',
  nacionalidad: 'Estados Unidos',
  edad: 24,
  patrimonio: 310000
}
{
  nombre: 'Camila Soto',
  nacionalidad: 'Chile',
  edad: 37,
  patrimonio: 350000
}
{
  nombre: 'Sebastian Vargas',
  nacionalidad: 'Colombia',
  edad: 30,
  patrimonio: 410000
}
{
  nombre: 'Alejandro Ramirez',
  nacionalidad: 'Mexico',
  edad: 32,
  patrimonio: 520000
}
{
  nombre: 'Yuki Sato',
  nacionalidad: 'Japon',
  edad: 35,
  patrimonio: 790000
}
{
  nombre: 'Martin Acosta',
  nacionalidad: 'Argentina',
  edad: 46,
  patrimonio: 830000
}
{
  nombre: 'Mariana Costa',
  nacionalidad: 'Brasil',
  edad: 44,
  patrimonio: 970000
}
}
myreplica-set [primary] Actor>

```

Figura 8: Captura que muestra el resultado del pipeline de agregación realizado en la shell.

Parte 2 - Procesamiento de consultas

1. Ejecución sin índices.

Primero se ejecutaron las mismas consultas de la sección anterior sin crear índices y haciendo uso de `.explain("executionStats")`. Los resultados se encuentran en el mismo github del inicio. En todos los casos MongoDB utilizó COLLSCAN (escaneo completo de la colección), ya que no existía ningún índice adicional al `_id`.

Consulta	Etapas raíz	docsExamined	keysExamined	nReturned	Tiempo (ms)
1. Edad > 50 (find)	COLLSCAN → SORT → PROJECTION	24	0	8	0
2. Nacionalidad="Chile" + Idiomas="Español" (find)	COLLSCAN → SORT → PROJECTION	24	0	6	0
3. Edad < 30 + Personajes \$elemMatch (Rol, Generos) (find)	COLLSCAN → SORT → PROJECTION	24	0	7	0
4. Ordenar por Patrimonio, skip 4, limit 10 (aggregate)	COLLSCAN → SORT → SKIP → PROJECT	24	0	10	0

Tabla 1: Comparaciones sobre los resultados de las consultas de la Parte 1 antes de agregar índices.

En las cuatro consultas totalDocsExamined es igual a 24, el total de documentos de la colección, ya que MongoDB tuvo que revisar todos los documentos, uno por uno, para evaluar el filtro, sin poder descartar ninguno de antemano. totalKeysExamined es 0 porque no se recorrió ninguna estructura de índice adicional al escaneo de la colección.

2. Índice sobre Nacionalidad.

Luego se crea un índice sobre Nacionalidad y se vuelve a ejecutar la consulta de la Parte 1 (consulta 2). El resultado se encuentra en github.

```
> db.getCollection("Voice-actors").createIndex(
  { Nacionalidad: 1 },
  { name: "idx_nacionalidad" }
)
< idx_nacionalidad
> db.getCollection("Voice-actors").getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { Nacionalidad: 1 }, name: 'idx_nacionalidad' }
]
```

Figura 9: Captura que muestra la creación del index sobre Nacionalidad en shell.

Métrica	Sin índice	Con idx_nacionalidad
Etapas de acceso	COLLSCAN	IXSCAN (idx_nacionalidad) → FETCH
docsExamined	24	6
keysExamined	0	6
nReturned	6	6
Tiempo (ms)	0	5

Tabla 2: Comparaciones sobre los resultados de la consulta 2 con y sin índice.

Con el índice, el plan ganador pasó de escanear toda la colección a recorrer directamente las entradas de `idx_nacionalidad` cuyo valor es "Chile". Esto redujo `docsExamined` de 24 a 6, ya que sólo se leyeron (mediante `FETCH`) los documentos cuya nacionalidad ya calzaba, y sobre esos 6 se aplicó en memoria el filtro restante de Idiomas. El aumento del tiempo reportado teniendo en cuenta que son solo 24 documentos, no se podría considerar una señal real de rendimiento.

3. Índice sobre Edad.

Luego se crea un índice sobre Edad y se vuelven a ejecutar las consultas de la Parte 1 (consultas 1 y 3). Los resultados se encuentran en github.

```
> db.getCollection("Voice-actors").createIndex(
  { Edad: 1 },
  { name: "idx_edad" }
)
< idx_edad
> db.getCollection("Voice-actors").getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { Nacionalidad: 1 }, name: 'idx_nacionalidad' },
  { v: 2, key: { Edad: 1 }, name: 'idx_edad' }
]
```

Figura 10: Captura que muestra la creación del index sobre Edad en shell.

Consulta 1 (Edad > 50):

Métrica	Sin índice	Con idx_edad
Etapas de acceso	COLLSCAN → SORT	IXSCAN (backward) → FETCH → LIMIT
docsExamined	24	8
keysExamined	0	8
nReturned	8	8
Tiempo (ms)	0	3

Tabla 3: Comparaciones sobre los resultados de la consulta 1 con y sin índice.

Consulta 3 (Edad < 30 + Personajes con Rol="Principal" y Generos="Accion"):

Métrica	Sin índice	Con idx_edad
Etapas de acceso	COLLSCAN → SORT	IXSCAN → FETCH (filtro \$elemMatch en memoria)
docsExamined	24	8
keysExamined	0	8
nReturned	7	7
Tiempo (ms)	0	0

Tabla 4: Comparaciones sobre los resultados de la consulta 3 con y sin índice.

En ambos casos `docsExamined` bajó de 24 a 8: el índice permitió ubicar directamente las 8 entradas cuya Edad cumple el rango (> 50 o < 30), en vez de revisar toda la colección. En la Consulta 3, el filtro sobre Personajes.\$elemMatch no está indexado, así que igualmente se necesita un `FETCH` que traiga el documento completo y aplique ese filtro en memoria, por lo que el índice solo aceleró la parte de Edad,

reduciendo los candidatos de 24 a 8 antes de aplicar el \$elemMatch. Con respecto a los tiempos, al ser muy pocos documentos, no se podrían considerar como medidas de rendimiento

4. Comparación de planes de ejecución

COLLSCAN vs IXSCAN: sin índices, MongoDB debe recorrer secuencialmente cada documento de la colección para evaluar el filtro (COLLSCAN), por lo que totalDocsExamined siempre fue igual al tamaño total de la colección (24). Al crear un índice sobre el campo usado en el filtro, el plan ganador cambia a IXSCAN: MongoDB recorre las entradas del índice dentro de los límites que satisfacen la condición, y solo mediante una etapa FETCH recupera los documentos completos que realmente calzan.

Claves examinadas (totalKeysExamined): pasó de 0 (no se usaba índice) a un valor igual a la cantidad de entradas del índice que caen dentro del rango consultado (6 para Nacionalidad="Chile"; 8 para Edad>50 y Edad<30).

Documentos examinados (totalDocsExamined): bajó de 24 en todas las consultas sin índice a un valor igual al número de claves examinadas en las consultas con índice (6 u 8 según el caso). Esto ocurre porque los índices creados son muy selectivos respecto al campo indexado, por lo que cada clave del índice corresponde a un documento que efectivamente se necesita traer.

Documentos retornados (nReturned): no cambió entre el antes y el después de crear cada índice (8, 6, 7 y 10 respectivamente). Esto es esperable ya que el índice sólo cambia el cómo MongoDB llega a los documentos, no cuáles documentos cumplen la consulta, por lo que el resultado debe ser igual.

Tiempo de ejecución (executionTimeMillis): con una colección de solo 24 documentos, los tiempos reportados (0, 3 o 5 ms) son demasiado pequeños para ser una métrica confiable, incluso se observó un caso (Consulta 2) donde el tiempo aumentó levemente al usar el índice.

Plan seleccionado por MongoDB: en todos los casos con índice, rejectedPlans aparece vacío, ya que al existir un único índice aplicable el planificador no tuvo alternativas entre las cuales competir; simplemente adoptó el plan que usa ese índice por sobre el COLLSCAN por defecto.

5. Escalabilidad

Si la colección pasara de aprox. 20 documentos a 10 mil millones:

Consultas sin índices: seguirían usando COLLSCAN, cuyo costo crece linealmente ($O(n)$) con el tamaño de la colección. Con 10 mil millones de documentos, cada consulta tendría que leer y evaluar potencialmente todos esos documentos, lo que en la práctica se traduciría en tiempos de ejecución de minutos, horas o más, además de requerir mucho cómputo, con alto riesgo de timeouts o saturación del servidor.

Consultas que pueden usar los índices creados (Nacionalidad, Edad): en vez de recorrer toda la colección, MongoDB buscaría directamente en la estructura del índice, cuyo costo de búsqueda crece de forma mucho más lenta respecto al tamaño total, más el costo de traer los documentos que efectivamente cumplen la condición. La diferencia de rendimiento frente al COLLSCAN se volvería mucho más drástica.

Cantidad de documentos examinados: sin índice, prácticamente el total de la colección (hasta 10 mil millones), mientras que con índice bien elegido, solo los documentos cuya clave indexada cae dentro del rango buscado, la cual es una fracción mucho menor.

Tiempo y recursos: sin índices, el tiempo y el consumo de I/O/CPU escalarían junto con el tamaño de la colección, haciendo inviables las consultas interactivas, mientras que con índices adecuados, el tiempo se mantendría acotado y crecería mucho más lento que el tamaño de la colección.

Almacenamiento adicional de los índices: cada índice ocupa espacio en disco proporcional al número de documentos y al tamaño del campo indexado. Con 10 mil millones de documentos, cada índice adicional

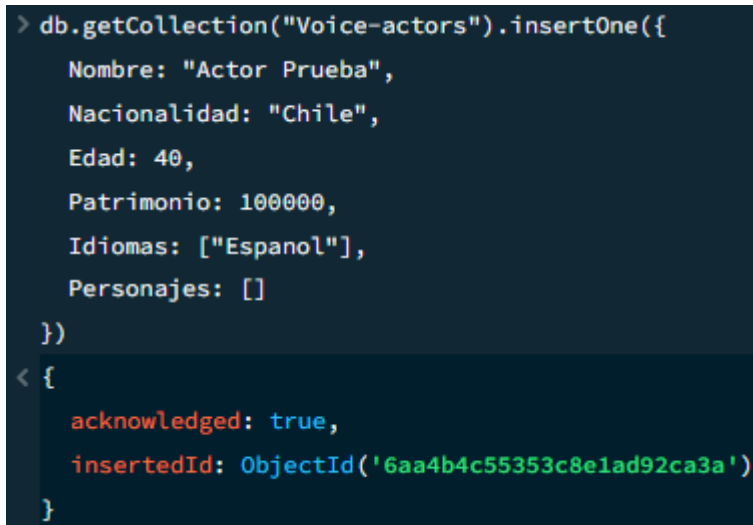
podría representar varios gigabytes o incluso terabytes, y además introduce un costo extra en cada operación de escritura (inserción, actualización o eliminación), ya que todos los índices existentes deben mantenerse actualizados.

En base a lo observado con `explain()` a pequeña escala, se puede proyectar que, a gran escala, la diferencia entre usar o no un índice deja de ser una simple optimización y pasa a ser determinante para que una consulta sea ejecutable en un tiempo razonable y sin consumir demasiado poder de cómputo. El diseño de índices se vuelve relevante porque permite evitar un costo lineal respecto al tamaño total de los datos, a cambio de espacio en disco adicional y de un pequeño costo extra en cada escritura.

Parte 3 - Flexibilidad de esquema, resiliencia y actualización

1. Flexibilidad del esquema

Se insertó un nuevo documento omitiendo intencionalmente el campo `fechaNacimiento`:



```
> db.getCollection("Voice-actors").insertOne({
  Nombre: "Actor Prueba",
  Nacionalidad: "Chile",
  Edad: 40,
  Patrimonio: 100000,
  Idiomas: ["Espanol"],
  Personajes: []
})
< {
  acknowledged: true,
  insertedId: ObjectId('6aa4b4c55353c8e1ad92ca3a')
}
```

Figura 11: Captura que muestra la inserción de un documento sin campo `fechaNacimiento`.

¿MongoDB permitiría insertar el documento?

Sí. Por defecto MongoDB no depende de esquema, por lo que cada documento de una colección puede tener una estructura distinta, y no exige que todos los campos estén presentes.

¿Qué consecuencias podría tener posteriormente?

Cualquier consulta, agregación o lógica de aplicación que asuma la existencia de `fechaNacimiento` (por ejemplo, para recalcular la edad, ordenar cronológicamente, etc.) podría fallar, arrojar `null/undefined`, o simplemente excluir ese documento de los resultados si se usa como filtro `FechaNacimiento`. También generaría inconsistencia de datos entre documentos "completos" e "incompletos".

¿Es conveniente permitir documentos con atributos faltantes en este caso?

No, porque `fechaNacimiento` es un dato base de identidad de la persona (según lo pedido en el enunciado como atributo mínimo), no un dato opcional o dependiente de contexto. Permitirlo faltante rompe la garantía de que todo documento de "Voice-actors" tiene la información mínima requerida.

¿Existe forma de controlar/validar la estructura?

Sí, mediante validadores de esquema aplicados a nivel de colección con `createCollection` o `collMod`, definiendo campos `required`, tipos y reglas adicionales. Esto permite imponer reglas mínimas de integridad cuando se requiera.

2. Resiliencia de la base de datos

Se verifica que el nodo `mongo1` sea el nodo primario y se detiene.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> docker exec -ti mongo1 bash
root@a81de1b364be:/# mongosh --port 30001
Current Mongosh Log ID: 6aa49560921d7adac3949971
Connecting to:      mongodb://127.0.0.1:30001/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:      8.0.30
Using Mongosh:       2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-09-11T04:08:04.122+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-09-11T04:08:04.122+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-11T04:08:04.122+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-09-11T04:08:04.122+00:00: We suggest setting the contents of sysfsFile to 0.
2026-09-11T04:08:04.122+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

my-replica-set [direct: primary] test> |
```

Figura 12: Captura que muestra la verificación del nodo primario.

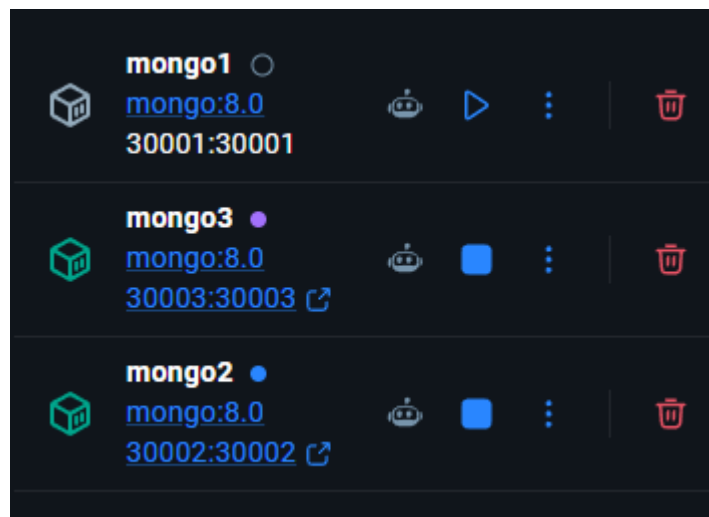


Figura 13: Captura que muestra que se detuvo el nodo mongo1 (primario).

```

lastHeartbeatMessage: 'Error connecting to mongo1:30001 (127.0.0.1:30001) :: caused by :: onInvoke :: caused by :: Connection refused',
syncSourceHost: '',
syncSourceId: -1,
infoMessage: '',
configVersion: 1,
configTerm: 2
},
{
  _id: 1,
  name: 'mongo2:30002',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 71790,
  optime: { ts: Timestamp({ t: 1789171447, i: 6 }), t: Long('2') },
  optimeDate: 2026-09-12T00:04:07.000Z,
  optimeWritten: { ts: Timestamp({ t: 1789171447, i: 6 }), t: Long('2') },
  optimeWrittenDate: 2026-09-12T00:04:07.000Z,
  lastAppliedWallTime: 2026-09-12T00:04:07.550Z,
  lastDurableWallTime: 2026-09-12T00:04:07.550Z,
  lastWrittenWallTime: 2026-09-12T00:04:07.550Z,
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  electionTime: Timestamp({ t: 1789171158, i: 1 }),
  electionDate: 2026-09-11T23:59:18.000Z,
  configVersion: 1,
  configTerm: 2,
  self: true,
  lastHeartbeatMessage: ''
},
{
  _id: 2,
  name: 'mongo3:30003',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 71756,
  optime: { ts: Timestamp({ t: 1789171447, i: 6 }), t: Long('2') }
}

```

Figura 14: Captura que muestra el nuevo nodo primario (mongo2).

{Edad:{\$gt:40}}

Explain Reset Find Options

ADD DATA BULK EXPORT DATA EXPORT CODE 25 1 - 12 of 12

	_id ObjectId	Nombre String	Nacionalidad String	FechaNacimiento String	Edad Int32	Patrimonio Int32	Idiomas Arri
1	ObjectId('6aa3864b805bfa...')	"Sarah Williams"	"Estados Unidos"	"1984-06-25"	42	2300000	[] 2 element
2	ObjectId('6aa3864b805bfa...')	"Mariana Costa"	"Brasil"	"1982-08-03"	44	970000	[] 3 element
3	ObjectId('6aa3864b805bfa...')	"Martin Acosta"	"Argentina"	"1980-01-09"	46	830000	[] 1 element
4	ObjectId('6aa3864b805bfa...')	"Carlos Navarro"	"Espana"	"1978-07-11"	48	1100000	[] 3 element
5	ObjectId('6aa3864b805bfa...')	"Kenji Nakamura"	"Japon"	"1974-09-29"	51	4600000	[] 2 element
6	ObjectId('6aa3864b805bfa...')	"Claire Dubois"	"Francia"	"1972-04-08"	54	2850000	[] 3 element
7	ObjectId('6aa3864b805bfa...')	"Gabriela Torres"	"Argentina"	"1970-09-15"	55	1450000	[] 2 element
8	ObjectId('6aa3864b805bfa...')	"Diego Morales"	"Chile"	"1971-06-05"	55	1200000	[] 2 element
9	ObjectId('6aa3864b805bfa...')	"Ricardo Sepulveda"	"Chile"	"1968-03-30"	58	1750000	[] 1 element
10	ObjectId('6aa3864b805bfa...')	"Patricia Contreras"	"Chile"	"1965-11-23"	60	2100000	[] 2 element
11	ObjectId('6aa3864b805bfa...')	"Roberto Mendoza"	"Mexico"	"1962-05-10"	64	3500000	[] 2 element
12	ObjectId('6aa3864b805bfa...')	"Michael Thompson"	"Estados Unidos"	"1960-02-18"	66	8200000	[] 1 element

Figura 15: Captura que muestra una consulta común realizada por la interfaz gráfica luego de detener el primario (mongo1) junto al resultado obtenido.

¿Funcionó la consulta?

Sí, la consulta se ejecutó correctamente contra el nuevo primario, retornando los resultados esperados.

¿Por qué?

Porque MongoDB, al operar como replica set de 3 nodos, mantiene los datos replicados en cada miembro. Al caer el primario, los 2 nodos restantes conservan la mayoría del quórum de votación, lo que les permite realizar automáticamente una elección y designar un nuevo primario en pocos segundos, sin pérdida de datos ni intervención manual demostrando la alta disponibilidad.

3. Actualización masiva de documentos

En esta sección se deben actualizar todos los documentos siguiendo estas indicaciones:

- Agregar una nueva producción para todos los actores y actrices (por simplicidad todos tienen el mismo personaje)
- Agregar el nuevo personaje al arreglo "Personajes" de todos los documentos.
- Para actores o actrices menores de 40 años, el personaje tendrá:
 - Rol: "principal".
 - Cantidad de apariciones: 3.
- Para actores o actrices de 40 años o más, el personaje tendrá:
 - Rol: "secundario".
 - Cantidad de apariciones: 1.
- El patrimonio de todos los actores y actrices deberá incrementarse en un 32 %

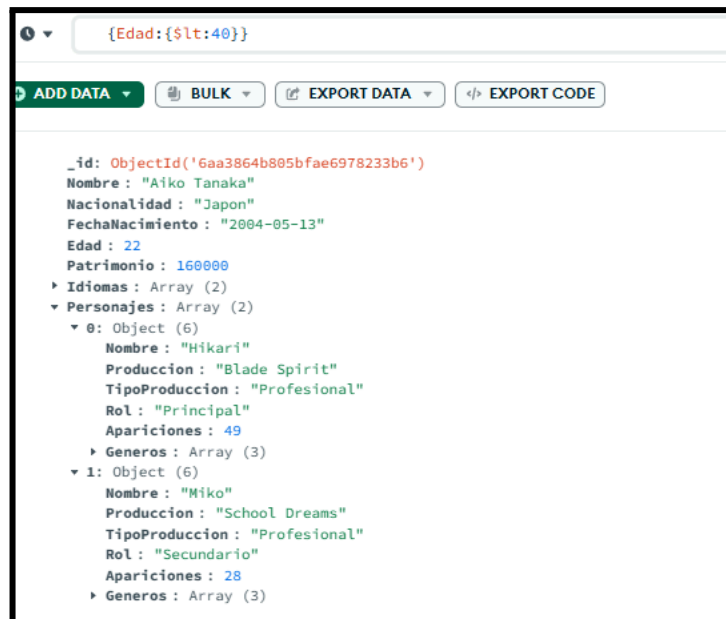


Figura 16: Captura que muestra una al primer resultado al ejecutar una consulta para ver los actores con edad menor a 40.

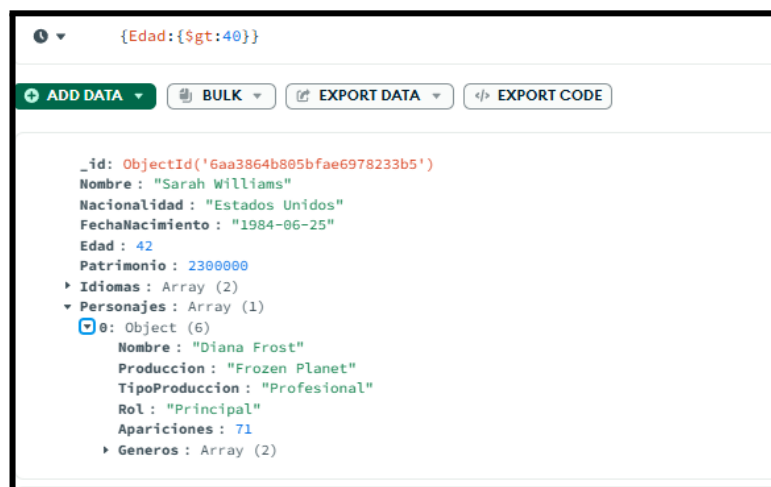


Figura 17: Captura que muestra una al primer resultado al ejecutar una consulta para ver los actores con edad mayor o igual a 40 (aquí cometí un error y puse gt en vez de gte, pero igualmente ningún actor de la colección tenía exactamente edad de 40, así que no afectó, corregido al hacer la actualización posterior).

```

> db.getCollection("Voice-actors").updateMany(
  {
    Edad: { $lt: 40 }
  },
  {
    $push: {
      Personajes: {
        Nombre: "Turbo",
        Produccion: "Sansa Ball Race",
        TipoProduccion: "Profesional",
        Rol: "Principal",
        Apariciones: 3,
        Generos: [
          "Animacion",
          "Comedia"
        ]
      }
    },
    $mul: {
      Patrimonio: 1.32
    }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 12,
  modifiedCount: 12,
  upsertedCount: 0
}

```

Figura 18: Captura que muestra la actualización correspondiente a los actores con edad menor a 40.

The screenshot shows a MongoDB query results interface. At the top, the query filter is `{Edad:{ $lt:40 }}`. Below the query, there are buttons for `ADD DATA`, `BULK`, `EXPORT DATA`, and `EXPORT CODE`. The results are displayed in a list format. The first result is for an actor named Aiko Tanaka, and the second result is for Luciana Fernandez. Both actors have their `Edad` (Age) updated to 22 and 23 respectively, and their `Patrimonio` (Wealth) updated to 211200 and 99000 respectively. The `Personajes` (Characters) array is also updated, showing the new character 'Turbo' added to the list.

```

{
  "_id": ObjectId('6aa3864b805bfae6978233b6'),
  "Nombre": "Aiko Tanaka",
  "Nacionalidad": "Japon",
  "FechaNacimiento": "2004-05-13",
  "Edad": 22,
  "Patrimonio": 211200,
  "Idiomas": Array (2),
  "Personajes": Array (3)
    0: Object (6)
    1: Object (6)
    2: Object (6)
      "Nombre": "Turbo",
      "Produccion": "Sansa Ball Race",
      "TipoProduccion": "Profesional",
      "Rol": "Principal",
      "Apariciones": 3,
      "Generos": Array (2)
  }
}

{
  "_id": ObjectId('6aa3864b805bfae6978233ab'),
  "Nombre": "Luciana Fernandez",
  "Nacionalidad": "Argentina",
  "FechaNacimiento": "2003-07-17",
  "Edad": 23,
  "Patrimonio": 99000,
  "Idiomas": Array (2),
  "Personajes": Array (3)
    0: Object (6)
    1: Object (6)
    2: Object (6)
      "Nombre": "Turbo",
      "Produccion": "Sansa Ball Race",
      "TipoProduccion": "Profesional",
      "Rol": "Principal",
      "Apariciones": 3,
      "Generos": Array (2)
  }
}

```

Figura 19: Captura que muestra los primeros resultados al ejecutar una consulta para ver los actores con edad menor a 40 actualizados (se muestra el mismo actor que en figura 16 y uno extra para demostrar que todos se actualizaron todos). Se puede observar que se agregó el nuevo personaje y que el patrimonio se encuentra actualizado.

```

> db.getCollection("Voice-actors").updateMany(
  {
    Edad: { $gte: 40 }
  },
  {
    $push: {
      Personajes: {
        Nombre: "Turbo",
        Produccion: "Sansa Ball Race",
        TipoProduccion: "Profesional",
        Rol: "Secundario",
        Apariciones: 1,
        Generos: [
          "Romance",
        ]
      }
    },
    $mul: {
      Patrimonio: 1.32
    }
  }
)
< {
  acknowledged: true,
  InsertedId: null,
  matchedCount: 12,
  modifiedCount: 12,
  upsertedCount: 0
}

```

Figura 20: Captura que muestra la actualización correspondiente a los actores con edad mayor o igual a 40.

The screenshot shows a MongoDB query result for the filter `{Edad:{$gte:40}}`. The results are displayed in a table-like format with expandable sections. The first result is for Sarah Williams, and the second is for Mariana Costa. Both actors have their age updated to 42 and 44 respectively, and their wealth (Patrimonio) updated to 3036600 and 1280400 respectively. The query filter was `{Edad:{$gte:40}}`.

Figura 21: Captura que muestra los primeros resultados al ejecutar una consulta para ver los actores con edad mayor o igual a 40 actualizados (se muestra el mismo actor que en figura 17 y uno extra para demostrar que todos se actualizaron todos). Se puede observar que se agregó el nuevo personaje y que el patrimonio se encuentra actualizado.

Parte 4 - Cierre

```

> db.dropDatabase()
< { ok: 1, dropped: 'Actors' }

```

Figura 22: Captura que muestra la eliminación de la base de datos utilizada.